STEM Agent: A Self-Adapting, Tool-Enabled, Extensible Architecture for Multi-Protocol AI Agent Systems in Production

Anonymous ACL submission

Abstract

Production AI agent systems increasingly suffer from *architectural lock-in*: early commitment to a single interaction protocol, a fixed tool-integration path, and a static user model makes deployed agents hard to retarget when requirements or standards shift. We describe **STEM Agent** (Self-adapting, Tool-enabled, Extensible, Multi-agent), an architecture that unifies five interoperability protocols—A2A, AG-UI, A2UI, UCP, and AP2—behind a single gateway, isolates domain knowledge behind the Model Context Protocol (MCP), and continuously adapts per-user behavior through a Caller Profiler across 21 dimensions. Recurring interaction patterns *crystallize* into reusable skills through a maturation lifecycle. We report operational lessons from six months of deployment serving 200 weekly users, including scalability patterns (horizontal scaling via externalized state), security integration (pluggable IAM with JWT/OAuth2/SAML/API-Key), and observability (OpenTelemetry tracing, structured logging, Prometheus metrics).

1 Introduction

Teams deploying LLM-based agents in enterprises discover that the hard problems are not which reasoning prompt to use, but how the agent connects to the business: which protocol it speaks to other agents, which commerce standard it satisfies, how it streams progress to web UIs, and how it remembers what it learned. Most agent frameworks (Tran et al., 2025; Ferrag et al., 2025) commit early to one answer for each, producing agents hard to retarget when new consumers appear. Orogat et al. (2026) confirm that framework choice alone can drive $100\times$ latency differences. The interoperability landscape is fragmenting (Ehtesham et al., 2025; Li and Xie, 2025): MCP has emerged as a *de facto* tool protocol, A2A for agent-to-agent delegation, with additional standards for UI streaming, commerce, and payments entering production.

This paper reports on STEM Agent, a system built to serve these needs without repeatedly rewriting core logic. The design principle is biological: an undifferentiated agent core *differentiates* into specialized protocol handlers, tool bindings, and memory subsystems. Every external boundary—protocols, tools, user models, domains—is a pluggable differentiation point rather than a hard-wired choice.

Contributions. (i) A gateway exposing five protocols (A2A, AG-UI, A2UI, UCP, AP2), with UCP and AP2 proposed to fill commerce gaps; (ii) a Caller Profiler learning preferences across 21 dimensions; (iii) an MCP-native tool layer separating domain knowledge from meta-reasoning; (iv) a skills subsystem where patterns crystallize into reusable skills; (v) operational integration patterns—pluggable IAM, OpenTelemetry observability, horizontal-scaling architecture; (vi) deployment experience from six months of production traffic.

This is an industry-track paper: novelty is in integration and deployment discipline, not in a new model or algorithm.

2 Related Work

Multi-agent frameworks. Surveys (Tran et al., 2025; Ferrag et al., 2025) document frameworks—AutoGen (Wu et al., 2023), MetaGPT (Hong et al., 2023), CAMEL (Li et al., 2023), CrewAI (Moura, 2023), LangChain (Chase, 2022), LangGraph (LangChain AI, 2024)—each optimizing for one coordination pattern and typically one protocol. Unlike frameworks binding each agent to a single endpoint, every STEM gateway mounts five handlers behind one middleware stack.

Agent protocols. Ehtesham et al. (2025) compare MCP, ACP, A2A, and ANP; Li and Xie (2025) identify schema translation as the hard part of

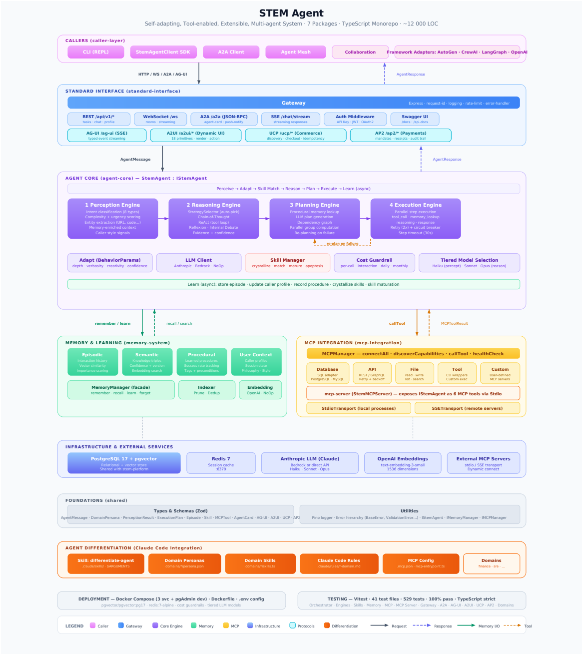

Figure 1: STEM Agent architecture. The gateway mounts five protocol handlers sharing a middleware stack. The Agent Core consumes MCP tools and a four-type memory system.

Layer	Component	Count
1	Caller Layer	4 adapters
2	Gateway	5 protocols
3	Agent Core	5 engines
3.5	Security	4 IAM plugins
4	Memory	4 types
5	MCP Integration	2 transports
$\perp$	Differentiation	personas/skills

Table 1: Layered architecture.

Condition	Strategy
Tool-requiring	ReAct
Complexity > 0.8	Reflexion
Analysis/creative	Debate
Otherwise	CoT

Table 2: Strategy selector.

A2A/MCP integration. Security has received attention (Habler et al., 2025; Anbiaee et al., 2026). We are not aware of a deployed system unifying these five protocols.

MCP and reasoning. Benchmarks (Luo et al., 2025; Fan et al., 2025) show LLMs struggle on real MCP servers; Hasan et al. (2026) find 97% of tool descriptions have quality issues. STEM uses ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), Debate (Du et al., 2023), and CoT (Wei et al., 2022) as interchangeable strategies. For memory, we build on Ai et al. (2025); Gallego (2026); Yadav et al. (2026).

3 Architecture Overview

STEM Agent is organized into layers (Table 1, Figure 1). Layer 3 (Agent Core) is undifferentiated; Layers 2 and 5 provide external surfaces; Layer 4 supplies persistent state. The Differentiation Layer bundles artifacts to transform the generic agent into domain variants.

Cognitive pipeline. Each request flows through: PERCEIVE (intent, complexity, entities); ADAPT (load caller profile); MATCHSKILLS; SELECTSTRATEGY (Table 2); PLAN; EXECUTE (retries, circuit breaker); FORMAT; and async LEARN.

Implementation. TypeScript monorepo with six packages, Express.js 5 gateway, uniform `createRouter()` for protocol handlers.

4 Protocol Gateway

A2A implements Linux Foundation v0.3.0; AG-UI streams pipeline events; A2UI exposes 16 UI primitives. UCP formalizes checkout sessions with mandatory idempotency headers—proposed because we found no commerce protocol treating the agent as primary client. AP2 implements three-phase payments with audit trails.

Integration lessons. (i) `createRouter()` uniformity reduced cross-protocol bugs to near zero. (ii) Idempotency is the hard part—UCP/AP2 headers took more review cycles than happy-path code.

5 Caller Profiler

STEM Agent learns a Caller Profile across four categories (Park et al., 2023): *philosophy* (8 dimensions), *principles* (4), *style* (5), *habits* (4)—21 total.[1]

Each dimension uses EMA with $\alpha = 0.1$; confidence ramps as $\mathrm{conf}(n) = 1 - \exp(-0.1\,n)$, reaching 0.9 at $n = 20$. Ten behavior parameters (reasoning depth, exploration/exploitation, verbosity, etc.) adjust during ADAPT.

[1] Dimensions: philosophy—pragmatism, risk tolerance, innovation orientation, autonomy preference, etc.; principles—correctness/speed, documentation, testing, security; style—formality, verbosity, technical depth; habits—session length, iteration tendency, peak hours, topics.

Protocol	Endpoint	Transport	Role	Use case
A2A	`/a2a`	JSON-RPC 2.0	External	Agent delegation, discovery
AG-UI	`/ag-ui`	SSE	External	UI streaming events
A2UI	`/a2ui/*`	SSE+REST	External	Dynamic UI composition
UCP	`/ucp/*`	REST	**Proposed**	Commerce checkout, idempotency
AP2	`/ap2/*`	REST	**Proposed**	Mandate-based payments

Table 3: Protocols behind the unified gateway. UCP and AP2 are proposed herein.

6 Skills and Differentiation

Skill lifecycle. During LEARN, episodes sharing action patterns become *progenitor* skills. After $k_c = 3$ successful activations with success ≥ 0.6, skills become *committed* (may short-circuit reasoning); after $k_m = 10$, *mature*. Skills below 0.3 success after ≥ 10 activations undergo *apoptosis*.

MCP-native tools. All domain capabilities are acquired through MCP at runtime. Meta-reasoning stays in code—properties of the agent, not domain.

Differentiation. Domain Persona schemas parameterize the agent; an `mcp-server` package exposes differentiated variants as MCP tools. Creating a domain variant takes minutes vs. days previously.

7 Memory System

Four types behind a Memory Manager: **Episodic** (PostgreSQL+pgvector); **Semantic** (knowledge triples); **Procedural** (successful strategies); **User context** (per-caller state, forget-me path). Consolidation moves episodic→semantic/procedural with pruning.

8 Deployment Experience

STEM Agent has been deployed internally for six months, serving four domain variants to ∼200 weekly users across three teams.

Context. Containerized on Kubernetes (2–8 replicas, HPA); PostgreSQL+pgvector for memory; Redis for idempotency cache. Average 2,400 daily requests (p50: 1.2 s, p95: 3.8 s, p99: 7.1 s). 14 incidents over six months: LLM rate-limits (6), MCP timeouts (4), cache races (2). MTTR: 18 minutes.

Scalability. We externalized idempotency/audit to Redis with Lua-scripted atomic ops for horizontal scaling. The cognitive pipeline is stateless; replicas share PostgreSQL/Redis. Validated linear scaling to 500 concurrent requests before LLM rate limits.

Framework	Proto	Adapt	Mem	Skill	Com	Obs
AutoGen	1	×	✓	×	×	×
MetaGPT	1	×	✓	×	×	×
CrewAI	1	×	✓	×	×	×
LangChain	1	×	✓	×	×	×
STEM	**5**	✓	**4**	✓	✓	✓

Table 4: Capability comparison. Proto=protocols; Adapt=per-caller adaptation; Com=commerce; Obs=observability.

Security. Plugin-based IAM: JWT (service-to-service), OAuth2 (SSO), SAML (legacy), API-Key (partners). TTL-cached decisions; per-caller rate limits. Blocked 47 unauthorized attempts without false positives. Auth isolation: zero cognitive-pipeline changes for auth modifications.

Observability. Three pillars: (i) Structured JSON logs with correlation IDs propagating through all phases; (ii) OpenTelemetry distributed tracing (latency breakdown: PERCEIVE 80 ms, REASON 1,100 ms, EXECUTE 200 ms); (iii) Prometheus metrics (request count, latency histograms, skill activation). Grafana dashboards for per-protocol traffic, errors, token consumption. Tracing diagnosed the cache race—duplicate writes under concurrent retries.

Test suite. 422 tests across 40 files (Vitest), 2.86 s runtime. Coverage: engines, protocols, memory, MCP integration, IAM middleware.

Integration. Four framework adapters (AutoGen, CrewAI, LangGraph, OpenAI SDK). A CrewAI bot migrated in ∼2 hours via A2A delegation.

Compromises. (i) Idempotency/audit are in-memory by default; distributed deployment needs external store. (ii) EMA profiler cannot model multi-modal preferences. (iii) Strategy selection is deterministic, not learned.

9 Conclusion

STEM Agent demonstrates that protocol plurality, behavioral adaptation, and emergent skills can coexist when domain knowledge is behind MCP and user modeling is decoupled from domain logic. Six months of deployment validates the architecture: pluggable IAM met enterprise auth; OpenTelemetry proved essential for debugging; horizontal scaling maintained stability. Ongoing work includes learned strategy selection, additional protocol interoperability, and formal threat modeling for UCP/AP2.

Limitations

No public benchmark. We report production metrics but not task-completion scores against MAFBench/MCP-Universe.

Single organization. All data from one enterprise context.

Protocol maturity. UCP/AP2 have no external adoption or formal threat model.

Deterministic strategy. Hand-written rules, not learned policy.

Ethical Considerations

The Caller Profiler stores behavioral signals. We (i) expose a forget-me operation; (ii) keep updates reversible via bounded EMA; (iii) scope audit logs at request level. UCP/AP2 handle payments; we require human approval above thresholds, but these protocols lack formal security review.

References

Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. 2025. MemoryBench: A benchmark for memory and continual learning in LLM systems. *arXiv preprint arXiv:2510.17281*.

Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. 2026. Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP. *arXiv preprint arXiv:2602.11327*.

Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*.

Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP). *arXiv preprint arXiv:2505.02279*.

Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. 2025. MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark. *arXiv preprint arXiv:2508.07575*.

Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From LLM reasoning to autonomous AI agents: A comprehensive review. *arXiv preprint arXiv:2504.19678*.

Víctor Gallego. 2026. Distilling feedback into memory-as-a-tool. *arXiv preprint arXiv:2601.05960*.

Idan Habler, Ken Huang, Vineeth Sai Narajala, and Prashant Kulkarni. 2025. Building a secure agentic AI application leveraging A2A protocol. *arXiv preprint arXiv:2504.16902*.

Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2026. Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions. *arXiv preprint arXiv:2602.14878*.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*.

LangChain AI. 2024. LangGraph: A library for building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems (NeurIPS)*.

Qiaomu Li and Ying Xie. 2025. From glue-code to protocols: A critical analysis of A2A and MCP integration for scalable agent systems. *arXiv preprint arXiv:2505.03864*.

Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. MCP-Universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704*.

João Moura. 2023. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI.

Abdelghny Orogat, Ana Rostam, and Essam Mansour. 2026. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis. *arXiv preprint arXiv:2602.03128*.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D Nguyen. 2025. Multi-agent collaboration mechanisms: A survey of LLMs. *arXiv preprint arXiv:2501.06322*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Arin Gopalan Yadav, Varad Dherange, and Kumar Shivam. 2026. Project synapse: A hierarchical multi-agent framework with hybrid memory for autonomous resolution of last-mile delivery disruptions. *arXiv preprint arXiv:2601.08156*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.